Binary Tree Data Structure

1) Introduction to Binary Tree and Their Construction

What is a Binary Tree?

A binary tree is a hierarchical data structure where each node has at most two children, referred to as the left child and right child. The topmost node is called the root, and nodes with no children are called leaf nodes.

Key Terminology

- **Root**: The topmost node in the tree
- **Parent**: A node that has one or more child nodes
- **Child**: A node that has a parent node
- **Leaf**: A node with no children
- **Height**: The longest path from root to any leaf node
- **Depth**: The distance from root to a particular node

Basic Node Structure

```java
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    // Constructor
    public TreeNode(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}
```

Constructing a Binary Tree

```java
public class BinaryTree {
    TreeNode root;

    public BinaryTree() {
        root = null;
    }

    // Example of manually creating a binary tree
    public void createTree() {
        root = new TreeNode(1);
        root.left = new TreeNode(2);
        root.right = new TreeNode(3);
        root.left.left = new TreeNode(4);
        root.left.right = new TreeNode(5);
    }
}
```

2) Application and Usage

Common Applications

File Systems: Directories and subdirectories are organized in a tree structure where each folder can contain files and other folders.

Expression Parsing: Mathematical expressions can be represented as binary trees where operators are internal nodes and operands are leaf nodes.

Database Indexing: Binary search trees are used in databases to maintain sorted data and allow fast searching.

Huffman Coding: Used in data compression algorithms where characters are represented as paths in a binary tree.

Decision Trees: Used in machine learning and artificial intelligence for making decisions based on multiple conditions.

Usage Example - Expression Tree

```java
// Representing expression: (a + b) * c
TreeNode root = new TreeNode('*');
root.left = new TreeNode('+');
root.right = new TreeNode('c');
root.left.left = new TreeNode('a');
root.left.right = new TreeNode('b');
```

3) Representation of Binary Tree and Their Construction

Array Representation

Binary trees can be represented using arrays where:

- Root is stored at index 0
- For node at index i:
  - Left child is at index (2*i + 1)
  - Right child is at index (2*i + 2)
  - Parent is at index (i-1)/2

```java
public class ArrayBinaryTree {
    int[] tree;
    int size;

    public ArrayBinaryTree(int capacity) {
        tree = new int[capacity];
        size = 0;
    }

    public void insert(int data) {
        if (size < tree.length) {
            tree[size] = data;
            size++;
        }
    }

    public int getLeftChild(int index) {
        int leftIndex = 2 * index + 1;
        return leftIndex < size ? tree[leftIndex] : -1;
    }

    public int getRightChild(int index) {
        int rightIndex = 2 * index + 2;
        return rightIndex < size ? tree[rightIndex] : -1;
    }
}
```

Linked Representation (Node-based)

```java
public class LinkedBinaryTree {
    TreeNode root;

    public LinkedBinaryTree() {
        root = null;
    }

    // Insert method for binary search tree property
    public void insert(int data) {
        root = insertRec(root, data);
    }

    private TreeNode insertRec(TreeNode root, int data) {
        if (root == null) {
            root = new TreeNode(data);
            return root;
        }

        if (data < root.data) {
            root.left = insertRec(root.left, data);
        } else if (data > root.data) {
            root.right = insertRec(root.right, data);
        }

        return root;
    }
}
```

Tree Traversal Methods

```java
// Inorder Traversal (Left, Root, Right)
public void inorder(TreeNode node) {
    if (node != null) {
        inorder(node.left);
        System.out.print(node.data + " ");
        inorder(node.right);
    }
}

// Preorder Traversal (Root, Left, Right)
public void preorder(TreeNode node) {
    if (node != null) {
        System.out.print(node.data + " ");
        preorder(node.left);
        preorder(node.right);
    }
}

// Postorder Traversal (Left, Right, Root)
public void postorder(TreeNode node) {
    if (node != null) {
        postorder(node.left);
        postorder(node.right);
        System.out.print(node.data + " ");
    }
}
```

4) Size, Sum, Maximum and Height of a Binary Tree

Finding the Size of a Binary Tree

The size is the total number of nodes in the tree.

```java
public int getSize(TreeNode node) {
    if (node == null) {
        return 0;
    }
    return 1 + getSize(node.left) + getSize(node.right);
}
```

Finding the Sum of All Nodes

```java
public int getSum(TreeNode node) {
    if (node == null) {
        return 0;
    }
    return node.data + getSum(node.left) + getSum(node.right);
}
```

Finding the Maximum Value

```java
public int getMaximum(TreeNode node) {
    if (node == null) {
        return Integer.MIN_VALUE;
    }

    int leftMax = getMaximum(node.left);
    int rightMax = getMaximum(node.right);

    return Math.max(node.data, Math.max(leftMax, rightMax));
}
```

Finding the Height of a Binary Tree

The height is the longest path from root to any leaf node.

```java
public int getHeight(TreeNode node) {
    if (node == null) {
        return -1; // Height of empty tree is -1
    }

    int leftHeight = getHeight(node.left);
    int rightHeight = getHeight(node.right);

    return 1 + Math.max(leftHeight, rightHeight);
}
```

Complete Example

```java
public class BinaryTreeOperations {
    public static void main(String[] args) {
        BinaryTree tree = new BinaryTree();

        // Create a sample tree
        tree.root = new TreeNode(10);
        tree.root.left = new TreeNode(5);
        tree.root.right = new TreeNode(15);
        tree.root.left.left = new TreeNode(3);
        tree.root.left.right = new TreeNode(7);
        tree.root.right.right = new TreeNode(20);

        // Perform operations
        System.out.println("Size: " + tree.getSize(tree.root));
        System.out.println("Sum: " + tree.getSum(tree.root));
        System.out.println("Maximum: " + tree.getMaximum(tree.root));
        System.out.println("Height: " + tree.getHeight(tree.root));
    }
}
```

Quiz Questions

1. What is the maximum number of children a node can have in a binary tree? a) 1 b) 2 c) 3 d) Unlimited

Answer: b) 2

2. In array representation of a binary tree, if a node is at index 3, where is its left child located? a) Index 6 b) Index 7 c) Index 8 d) Index 9

Answer: b) Index 7 (2*3 + 1 = 7)

3. What is the time complexity of finding the height of a binary tree? a) O(1) b) O(log n) c) O(n) d) $O(n^2)$

Answer: c) O(n)

4. In which traversal method do we visit the root node first? a) Inorder b) Preorder c) Postorder d) Level order

Answer: b) Preorder

5. What is the height of a binary tree with only one node (root)? a) -1 b) 0 c) 1 d) 2

Answer: b) 0

Practice Coding Questions

1. Write a Java method to count the number of leaf nodes in a binary tree.

2. Implement a method to check if two binary trees are identical (same structure and same values).

3. Create a method to find the minimum value in a binary tree.

4. Write a function to perform level-order traversal (breadth-first search) of a binary tree and print the nodes level by level.

5. Implement a method to check if a binary tree is a valid binary search tree (BST).